NightFuel | Project Design Document v2.0

NIGHT FUEL

Chrono-Nutrition & Fitness Platform for Shift Workers

Version	Date	Stack	Architecture
2.0	February 2026	TypeScript 5.5 + Python 3.12 + Next.js 15	14 Microservices + Turborepo Monorepo

Version 2.0 | Last Updated: February 2026

Table of Contents

1. [Executive Summary](#1-executive-summary)
2. [Problem Statement](#2-problem-statement)
3. [Target Users](#3-target-users)
4. [Core Features](#4-core-features)
5. [System Architecture](#5-system-architecture)
6. [Technology Stack](#6-technology-stack)
7. [Microservices — Detailed Breakdown](#7-microservices--detailed-breakdown)
8. [AI Architecture (3-Layer Model)](#8-ai-architecture-3-layer-model)
9. [Data Models](#9-data-models)
10. [Web Client — Pages & Flows](#10-web-client--pages--flows)
11. [Food Database Strategy](#11-food-database-strategy)
12. [Security Architecture](#12-security-architecture)
13. [Database & Migration Strategy](#13-database--migration-strategy)
14. [Event-Driven Communication](#14-event-driven-communication)
15. [Deployment Architecture](#15-deployment-architecture)
16. [Environment Variables Reference](#16-environment-variables-reference)
17. [Development Workflows](#17-development-workflows)
18. [Roadmap](#18-roadmap)

1. Executive Summary

NightFuel is a production-grade, enterprise-level chrono-nutrition and fitness platform designed specifically for **shift workers** — nurses, security guards, factory workers, call centre agents, truck drivers, and anyone whose work schedule disrupts normal circadian rhythms.

While mainstream fitness apps (MyFitnessPal, Fitbit, etc.) are built for people who sleep 11 PM–7 AM and work 9 AM–5 PM, NightFuel is built for the estimated **1.8 billion shift workers worldwide** whose biology operates on completely different timing.

The platform combines:

- A **deterministic Python circadian model** that computes each user's unique biological clock from their shift schedule
- **Chrono-nutrition science** — aligning meal types, macros, and timing to circadian phases
- A **3-layer AI system** using Anthropic Claude to generate highly personalised, adaptive plans
- A **dual food database** — 3M+ global foods plus an offline whole-foods library

2. Problem Statement

The Challenge

Shift workers face compounded health risks compared to day workers:

- **23% higher risk** of type 2 diabetes (misaligned insulin sensitivity windows)
- **29% higher risk** of cardiovascular disease (chronic circadian disruption)
- **Severely impacted** metabolism — same food eaten at the wrong circadian phase has different metabolic effects
- **Poor food choices** at night — no appropriate nutrition guidance for night-time eating
- **Workout scheduling impossible** — standard workout apps assume daytime energy peaks

What Existing Apps Get Wrong

Problem	How Current Apps Handle It	How NightFuel Handles It
Meal timing	Fixed 3-meal structure (B/L/D)	Reverse-engineered from sleep window
Workout scheduling	Morning/evening presets	Computed from shift type, fatigue, circadian phase
Food quality at night	Generic calorie counting	Night-appropriate foods (easily digestible, sleep-friendly)
Caffeine	Ignored	Precise cut-off windows per circadian phase
Energy management	Not tracked	Fatigue-integrated intensity adjustment

3. Target Users

Primary Segment: Shift Workers

- **Healthcare**: Nurses, doctors, paramedics (rotating 12-hour shifts)
- **Security & Law enforcement**: Night guards, police officers
- **Manufacturing & Logistics**: Factory workers, warehouse staff, truck drivers
- **Hospitality**: Hotel staff, restaurant workers, bartenders
- **Tech & Call Centres**: Customer support agents, network operations

Secondary Segment: Premium Users

- **Coaches** — Certified fitness and nutrition coaches who manage shift-worker clients
- **Athletes** — Bodybuilders and strength athletes who track periodisation and macros precisely
- **Health-conscious general users** — who want detailed nutrition tracking and AI coaching

User Personas

"Night Nurse Nadia" — 34, works 3×12h night shifts per week in a hospital ICU. Eats at 2 AM, 5 AM, and struggles with weight gain despite counting calories. Needs a plan that respects her sleep window (8 AM–4 PM) and recommends the right foods for the right phases of her shift.

"Rotating Raza" — 28, works a 2-week day / 2-week night rotating schedule at a factory. Needs a system that adapts his entire workout and meal plan every two weeks as his schedule rotates.

"Coach Carlos" — 42, certified personal trainer with 12 night-shift clients. Needs a dashboard to manage all clients, create custom plans, and track their adherence.

---

4. Core Features

4.1 User & Profiles

- **Multi-step onboarding**: age, gender, height, weight, fitness goals, experience level
- **Lifestyle & Sleep input**: night-worker toggle, exact sleep schedule, custom routine setup
- **Auto-calculations**: BMI, BMR (Mifflin-St Jeor), TDEE (Harris-Benedict × activity multiplier)
- **Shift scheduling**: regular, rotating, variable, on-call patterns with rotation period support
- **Dietary preferences**: vegan, vegetarian, keto, paleo, halal, budget-friendly, acne-safe, Ramadan mode

4.2 AI Nutrition

- **Night-shift optimised meal timing**: Pre-shift, mid-shift, recovery, and sleep-prep meals
- **Reverse meal planner**: Constructs meal schedule backwards from user's sleep window
- **Circadian-aligned macros**: Protein timing, carb manipulation, fat type recommendations per phase
- **Grocery list generation**: Auto-generated from the week's plan, budget-filtered
- **Acne-safe mode**: Eliminates high-glycaemic and dairy-heavy foods
- **Ramadan mode**: Adjusts to Suhoor/Iftar structure with hydration reminders
- **Weekly adaptive optimisation**: AI adjusts next week's plan based on adherence data

4.3 AI Workout Recommendation

- **Personalised plans**: Home, gym, or hybrid; equipment-aware
- **Bodybuilder mode**: Hypertrophy-focused, progressive overload with deload weeks
- **Split routines**: PPL (Push-Pull-Legs), Bro split, Full body, Upper-Lower
- **Night-shift-friendly scheduling**: Avoids high-intensity workouts within 3 hours of sleep
- **Smart rest day recommendations**: Based on fatigue score + sleep quality + shift intensity
- **Intensity auto-adjustment**: Fatigue and sleep metrics drive session intensity
- **Time-based sessions**: 20 / 40 / 60 minute options
- **Beginner-friendly progression**: Linear progression for new lifters
- **Injury-safe recommendations**: Avoids exercises contraindicated by reported injuries

4.4 Night-Shift Optimisation (Unique Selling Feature)

- **Circadian rhythm model**: Python engine computes from shift start/end, sleep window, historical data
- **Meal window calculator**: Identifies optimal windows for each meal type based on melatonin/cortisol curves
- **Caffeine timing guidance**: Precise cut-off time computed per user's circadian phase
- **Sleep quality optimisation**: Meal type and timing recommendations for better daytime sleep
- **Energy management**: Tracks fatigue trajectory across shift hours

4.5 Tracking & Analytics

- **Calorie & macro logging**: Daily targets vs actuals with macro dials
- **Weight & body composition tracking**: With trend charts
- **Strength tracking**: Personal records, 1RM estimates (Brzycki formula)
- **Sleep logging**: Duration, quality (1-10), circadian alignment score
- **Water intake tracking**
- **Adherence streaks**: Consecutive days on-plan
- **AI weekly performance reports**: Insight text generated by Claude from the week's data

4.6 Advanced Features

- **Coach Dashboard**: Client management, custom plan creation, direct messaging
- **Community**: Forums, progress sharing, team challenges
- **AI Assistant**: 24/7 chat coach with full user data context (Claude Haiku)
- **Subscriptions**: Free / Pro / Coach tiers (Stripe-powered)
- **Notifications**: Meal timing alerts, caffeine cut-off reminders, workout start prompts

5. System Architecture

High-Level Overview

```
┌──────────────────────────────────────────────────────────────┐
│  CLIENTS                                                     │
│  Next.js 15 (web)          React Native / Expo (mobile)      │
```

```
└──────────────────────────────┬──────────────────────────────┘
                               │ HTTPS / WSS
┌──────────────────────────────▼──────────────────────────────┐
│  API GATEWAY (Next.js rewrites)                              │
│  /api/auth → :3001  /api/meals → :3006  /api/chat → :3014 ...  │
└──┬────┬────┬────┬────┬────┬────┬────┬──────────┘
   │    │    │    │    │    │    │    │
  :3001 :3002 :3005 :3006 :3007 :3009 :3011 :3013 ...
 auth  shift  plan  meal  prog  user  ex  comm
   │    │    │    │    │    │    │
   └────┴────┴────┴────┴────┴────┘
                    │
            ┌───────▼────────┐
            │   Redis Pub/Sub    │  (Event Bus)
            │  (Upstash Redis 7) │
            └────────┬───────┘
                     │
        ┌────────────▼────────────┐
        │ Python AI Services      │
        │ :3003 circadian-engine  │
        │ :3004 ai-pipeline (Claude) │
        └────────────────────────┘
                     │
            ┌───────▼────────┐
            │ PostgreSQL 16      │
            │ (Supabase)         │
            │ One schema per svc │
            └────────────────┘
```

Communication Patterns

Pattern	When Used	Technology
Async events	After state changes (meal logged, shift created, plan generated)	Redis Pub/Sub via `@nightfuel/events`
Sync HTTP	When a service needs data from another to fulfil a request	Axios with 3s timeout + circuit breaker
WebSocket	Real-time coach chat	`@fastify/websocket`
SSE	AI streaming responses	FastAPI `StreamingResponse`

6. Technology Stack

Backend

Technology	Version	Purpose
Node.js	22 LTS	Runtime for all TS services
Fastify	5.x	HTTP framework (TS services)
TypeScript	5.5+ (strict)	Language
Turborepo	2.x	Monorepo task orchestration
Prisma	6.x	ORM + migrations
PostgreSQL	16	Primary database (via Supabase)
Redis	7 (Upstash)	Cache + event bus
ioredis	5.x	Redis client
Zod	3.x	Runtime validation + type inference
Pino	9.x	Structured JSON logging
fastify-type-provider-zod	Latest	Zod integration for Fastify

AI & Python Services

Technology	Version	Purpose
Python	3.12	AI services runtime
FastAPI	0.110+	HTTP framework (Python services)
numpy	1.26+	Circadian model calculations
Anthropic SDK	Latest	Claude API client

Frontend

Technology	Version	Purpose
Next.js	15 (App Router)	Web client
React	19	UI framework
Tailwind CSS	3.x	Styling
Framer Motion	11.x	Animations
TanStack Query	5.x	Server state management
Zustand	4.x	Client state management
Axios	1.x	HTTP client
Lucide React	Latest	Icons
Recharts	2.x	Data visualisation
Sonner	Latest	Toast notifications

7. Microservices — Detailed Breakdown

auth-service (:3001)

Responsibility: User authentication and session management

Endpoints:

- `POST /v1/auth/register` — create account (email + password hash)
- `POST /v1/auth/login` — returns JWT access token + refresh token
- `POST /v1/auth/refresh` — rotate refresh token
- `POST /v1/auth/logout` — invalidate refresh token
- `GET /v1/auth/me` — current user from JWT

Events published: `nightfuel:auth:user-registered`

shift-service (:3002)

Responsibility: Shift schedule management and rotation patterns

Endpoints:

- `POST /v1/shifts` — create a shift

- `GET /v1/shifts` — list user's shifts (with date range filter)
- `GET /v1/shifts/:id` — get single shift
- `POST /v1/shifts/rotation` — create rotation pattern
- `GET /v1/shifts/rotation/next` — preview next N days of rotation

Events published: `nightfuel:shift:shift-created`, `nightfuel:shift:rotation-updated`

circadian-engine (:3003) — Python/FastAPI

Responsibility: Deterministic circadian rhythm modelling

Algorithm:

19. Takes shift start/end, sleep window, timezone, historical sleep data
20. Models melatonin onset (2 hours before habitual sleep time)
21. Models cortisol peak (1 hour after habitual wake time)
22. Computes insulin sensitivity peak (morning anchor of circadian cycle)
23. Generates meal windows with recommended calorie percentages
24. Outputs caffeine and exercise windows

Endpoints:

- `POST /v1/circadian/compute` — compute full circadian profile for a shift day
- `GET /v1/circadian/profile/:userId` — latest cached profile

ai-pipeline (:3004) — Python/FastAPI

Responsibility: Claude API integration — plan generation, meal scoring, weekly reports

3-Layer AI Model:

```
Layer 1: circadian-engine (Python/numpy) — deterministic, fast, free
Layer 2: Rule engine — hard constraints (allergies, halal, macro floors)
Layer 3: Claude API — fills the JSON skeleton with personalised content
```

Endpoints:

- `POST /v1/ai/day-plan` — generate full day plan (meals + workout)
- `POST /v1/ai/meal-swap` — swap one meal for a better alternative
- `POST /v1/ai/weekly-report` — generate AI insights for the week
- `POST /v1/ai/chat` — AI assistant chat (streaming SSE)

Cost optimisation: Layer 1 and 2 produce a structured JSON skeleton. Claude only fills in text content (descriptions, tips, titles), not macro calculations. This reduces token usage by ~70%.

plan-service (:3005)

Responsibility: Day plan storage and retrieval, meal window scheduling

Endpoints:

- `POST /v1/plans` — create or regenerate today's plan
- `GET /v1/plans/today` — today's plan
- `GET /v1/plans/:date` — plan for a specific date

- `GET /v1/plans/:userId/meals` — meal schedule for grocery list

Events subscribed: `nightfuel:shift:shift-created` → auto-generates new plan

meal-service (:3006)

Responsibility: Food logging, local food database, recipes, fasting tracker

Food search — dual source:

- `GET /v1/meals/search?q=kale` — searches local FoodDB (760 whole foods, offline)
- Web client `/api/food-search` — proxies to Open Food Facts (3M+ foods, online)

Endpoints:

```
GET  /v1/meals/search          Local FoodDB search (filter: foodGroup, isVegan, isHalal, isGlutenFree)
GET  /v1/meals/food/:id        Single food by ID
GET  /v1/meals/food-groups     All food group names (for filter dropdowns)
POST /v1/meals/log             Log a meal with food items
GET  /v1/meals/logs            Meal history (filter by date)
GET  /v1/meals/grocery-list    Generate grocery list from plan-service
GET  /v1/meals/recipes         Browse recipes (filter by tags)
POST /v1/meals/recipes         Create a recipe
GET  /v1/meals/recipes/:id     Single recipe
GET  /v1/meals/fasting         Fasting log history
POST /v1/meals/fasting/start   Start a fasting period (target hours: 16)
POST /v1/meals/fasting/end     End active fasting period
```

Events published: `nightfuel:meal:meal-logged`

Database tables: `food_items`, `meal_logs`, `recipes`, `fasting_logs`

progress-service (:3007)

Responsibility: Body metrics, adherence tracking, streaks, weekly reports

Endpoints:

- `POST /v1/progress/metrics` — log body measurements
- `GET /v1/progress/metrics` — history with trend data
- `GET /v1/progress/streak` — current and longest streaks
- `GET /v1/progress/report/weekly` — AI-generated weekly summary

Events subscribed: `nightfuel:meal:meal-logged` → updates daily calorie actuals and adherence

user-service (:3009)

Responsibility: User profiles, onboarding, dietary preferences

Endpoints:

- `GET /v1/users/profile` — get profile
- `PUT /v1/users/profile` — update profile
- `GET /v1/users/preferences` — get dietary/workout preferences

- `PUT /v1/users/preferences` — update preferences
- `POST /v1/users/onboarding/complete` — mark onboarding done

exercise-service (:3011)

Responsibility: Workout logging, ExerciseDB integration, 1RM tracking

Endpoints:

- `GET /v1/exercises/search` — ExerciseDB search with caching
- `POST /v1/exercises/workout` — log a workout
- `GET /v1/exercises/history` — workout history
- `GET /v1/exercises/1rm/:name` — personal record for an exercise

chat-service (:3014)

Responsibility: Real-time coach chat + AI assistant

Protocol: WebSocket (`@fastify/websocket`) for live messages, REST for history

Endpoints:

- `GET /v1/chat/conversations` — list conversations
- `GET /v1/chat/conversations/:id/messages` — message history
- `WS /v1/chat/ws` — real-time message channel
- `GET /v1/chat/coaches` — coach directory (marketplace)

---

8. AI Architecture (3-Layer Model)

```
User Request: "Generate my meal plan for tonight's 10 PM–6 AM shift"
        |
        ▼
┌──────────────────────────────────────────┐
|  LAYER 1: circadian-engine (Python + numpy)    |
|                                                |
|  Input:  shift schedule, sleep window, tz      |
|  Output: melatonin onset, cortisol peak,       |
|          meal windows with calorie %,           |
|          caffeine cut-off, exercise window     |
|                                                |
|  Cost: $0 | Latency: <50ms                    |
└──────────────────┬───────────────────────────┘
                   | Circadian Profile
                   ▼
┌──────────────────────────────────────────┐
|  LAYER 2: Rule Engine (Node.js / TypeScript)   |
|                                                |
```

```
│  Hard constraints applied:                    │
│  - Allergen exclusions                         │
│  - Halal / vegan / GF filter                  │
│  - Macro floors (min protein, max sat fat)    │
│  - Night-eating rules (no heavy fats 2h pre-  │
│    sleep, easily digestible pre-shift)         │
│  - Budget cap applied to food selection        │
│                                                │
│  Output: Structured JSON skeleton with        │
│          meal slots, macro targets, constraints│
│                                                │
│  Cost: $0 | Latency: <20ms                    │
└──────────────────┬──────────────────────────┘
                   │ JSON Skeleton
                   ▼
┌──────────────────────────────────────────────┐
│  LAYER 3: Claude API (Anthropic)               │
│                                                │
│  Model: claude-sonnet-4-5 (plans)              │
│         claude-haiku (quick responses, chat)   │
│                                                │
│  Task: Fill in the skeleton with:             │
│  - Specific food suggestions per meal          │
│  - Prep instructions                           │
│  - Rationale ("This meal is high in tryptophan│
│    to support melatonin production before your │
│    sleep window at 7 AM")                      │
│  - Weekly insight narrative                    │
│                                                │
│  Cost optimisation:                           │
│  - Skeleton reduces Claude input tokens ~70%  │
│  - Haiku for all non-plan interactions        │
│  - Redis cache on identical skeleton hashes   │
│                                                │
│  Latency: 1–3s (Sonnet) / 200ms (Haiku)      │
└──────────────────────────────────────────────┘
```

9. Data Models

FoodItem

```typescript
interface FoodItem {
  id:            string;       // UUID
  name:          string;
  calories:      number;       // kcal per 100g
  protein:       number;       // g per 100g
  carbs:         number;       // g per 100g
  fat:           number;       // g per 100g
  fiber:         number;       // g per 100g
  sugar:         number;       // g per 100g
  sodiumMg:      number;       // mg per 100g
  servingSize:   string;       // e.g. "100g", "1 cup (240ml)"
  glycemicIndex: number | null;
  isVegan:       boolean;
  isGlutenFree:  boolean;
  isHalal:       boolean;
  region:        string | null;
  cuisineTags:   string[];     // e.g. ["mediterranean", "middle-eastern"]
  source:        string;       // 'FOODB' | 'OPENFOODFACTS' | 'CUSTOM'
  foodGroup:     string | null; // 'Fruits', 'Vegetables', 'Meats', ...
  createdAt:     Date;
  updatedAt:     Date;
}
```

MealLog

```typescript
interface MealLog {
  id:             string;
  userId:         string;
  loggedAt:       Date;
  mealType:       'BREAKFAST' | 'LUNCH' | 'DINNER' | 'SNACK';
  foodItems:      LoggedFoodItem[];   // JSON array
  totalCalories:  number;
  totalProtein:   number;
  totalCarbs:     number;
  totalFat:       number;
  isAdherent:     boolean;
  createdAt:      Date;
```

```
  updatedAt:      Date;
}
```

CircadianProfile

```typescript
interface CircadianProfile {
  userId:                  string;
  shiftId:                 string;
  computedAt:              string;   // ISO timestamp
  sleepStart:              string;   // "07:30"
  sleepEnd:                string;   // "15:30"
  caffeineWindowStart:     string;
  caffeineWindowEnd:       string;   // Cut-off time
  exerciseWindowStart:     string;
  exerciseWindowEnd:       string;
  melatoninOnset:          string;
  cortisolPeak:            string;
  insulinSensitivityPeak:  string;
  mealWindows:             MealWindow[];
}
```

UserPreferences

```typescript
interface UserPreferences {
  userId:                  string;
  primaryGoal:             'LOSE_FAT' | 'BUILD_MUSCLE' | 'MAINTAIN' | 'RECOMPOSE';
  dietaryPreference:       'STANDARD' | 'VEGAN' | 'VEGETARIAN' | 'KETO' | 'PALEO' | 'HALAL';
  targetCalories:          number | null;   // Auto-calculated from TDEE
  targetProteinG:          number | null;
  targetCarbsG:            number | null;
  targetFatG:              number | null;
  activityLevel:           'SEDENTARY' | 'LIGHTLY_ACTIVE' | 'MODERATELY_ACTIVE' | 'VERY_ACTIVE';
  experienceLevel:         'BEGINNER' | 'INTERMEDIATE' | 'ADVANCED';
  lifestyleType:           'DAY_WORKER' | 'NIGHT_WORKER' | 'ROTATING' | 'IRREGULAR';
  allergies:               string[];
  healthConditions:        string[];
  dietMode:                'STANDARD' | 'RAMADAN' | 'ACNE_SAFE' | 'BUDGET';
  workoutEnvironment:      'HOME' | 'GYM' | 'HYBRID' | null;
  availableEquipment:      string[];
  isBodybuilderMode:       boolean;
  isInjurySafeMode:        boolean;
  workoutDurationPreference: 20 | 40 | 60;
```

```
  splitPreference:         'PPL' | 'BRO_SPLIT' | 'FULL_BODY' | 'UPPER_LOWER';
}
```

10. Web Client — Pages & Flows

Route Map

```
/                            Landing page
/login                       Login form
/register                    Registration
/onboarding                  Multi-step onboarding wizard
|
/dashboard                   Main dashboard (daily summary)
|
├── /dashboard/meals         Meal Logger (food search + plate builder)
|   ├── /encyclopedia        Food Encyclopedia (3M+ foods via OFF)
|   ├── /recipes             Recipe browser
|   └── /planner             Circadian meal planner
|
├── /dashboard/exercises     Exercise Library (ExerciseDB)
|   ├── /history             Workout history
|   ├── /routines            Custom routines
|   ├── /calculator          1RM / BMR / TDEE calculators
|   └── /analytics           Strength progress charts
|
├── /dashboard/plan          Today's AI plan (meals + workouts)
├── /dashboard/sleep         Sleep log + circadian alignment
|
├── /dashboard/performance   Body metrics hub
|   ├── /measurements        Weight, body fat, measurements
|   ├── /photos              Progress photos
|   └── /reports             AI weekly reports
|
├── /dashboard/coaches       Coach marketplace
├── /dashboard/community     Forums, challenges
|
├── /settings/profile        Profile settings
├── /settings/preferences    Dietary & workout preferences
├── /settings/subscription   Billing & tier management
└── /settings/notifications  Push notification preferences
```

Meal Logger Flow

```
User opens /dashboard/meals
        |
        ▼
[Toggle:   Online |   Library]
        |
  ┌─────┴─────┐
  |           |
Online     Library
  |           |
  ▼           ▼
/api/food-  /api/meals/search
search      (meal-service local DB)
(OFF API)   (760 whole foods)
  |           |
  └─────┬─────┘
        |
        ▼
FoodResultCard (name, macros, dietary badges, expandable micronutrients)
        |
User clicks [+]
        |
        ▼
LogEntry added to "Your Plate"
(quantity controls, live macro dials)
        |
User clicks [Log Meal]
        |
        ▼
POST /api/meals/log → meal-service → DB
Event: nightfuel:meal:meal-logged → Redis
        |
progress-service subscribes → updates daily targets
        |
Redirect to /dashboard
```

11. Food Database Strategy

Sources

Source	Size	Type	Update Frequency	In DB
FoodDB (FOODB)	760 foods	Whole foods (scientific dataset)	One-time seed	✅ Seeded
Open Food Facts	3M+ products	Branded & packaged foods	Live API	API only
Custom	User-created	User-entered recipes/foods	On create	Future

FoodDB Seeding Process

The `extract-foodb.py` script:

25. Reads `foodb_2020_04_07_json.zip` (NDJSON format)
26. Joins `Food.json` + `Content.json` + `Nutrient.json`
27. Maps nutrient IDs: `{1: fat, 2: protein, 3: carbs, 5: fiber, 38: calories}`
28. Converts units: Energy in `kcal/100g`; macros in `mg/100g` → divide by 1000 for grams
29. Uses **median** (not mean) across multiple readings per nutrient to handle outliers
30. Auto-assigns dietary flags from food group (e.g. all "Fruits" → isVegan=true)
31. Outputs 760 validated records (232 skipped for insufficient data)

Open Food Facts Integration

- **Endpoint**: `GET /api/food-search?q=kale&region=world&limit=20`
- **Barcode**: `GET /api/food-search?barcode=3017624010701`
- **SSRF protection**: Only `ALLOWED_REGIONS` list of OFF subdomains are permitted
- **Caching**: 86,400s (24h) for barcode lookups; 3,600s (1h) for text search
- **Micronutrients returned**: vitamins C/A/D/B6/B12, folate, calcium, iron, potassium, magnesium, zinc, phosphorus

12. Security Architecture

Authentication Flow

```
1. User logs in → auth-service issues JWT (15min) + refresh token (7d)
2. JWT stored in HTTP-only cookie (nf_auth)
3. All API requests include cookie (automatic, httpOnly)
4. Each service validates JWT via fastify.authenticate preHandler
5. Refresh via POST /api/auth/refresh → rotates both tokens
```

Route Protection (Next.js)

`proxy.ts` middleware runs on every request:

- `nf_auth` cookie present + hitting auth routes → redirect to `/dashboard`
- `nf_auth` cookie absent + hitting protected routes → redirect to `/login?redirect=<path>`

Security Headers (next.config.js)

Header	Value
`X-Frame-Options`	`DENY`
`X-Content-Type-Options`	`nosniff`
`Strict-Transport-Security`	`max-age=63072000; includeSubDomains; preload`
`Content-Security-Policy`	Strict allowlist (self + Supabase + OpenFoodFacts + ExerciseDB only)
`Permissions-Policy`	Disables camera, microphone, geolocation, payment
`Referrer-Policy`	`strict-origin-when-cross-origin`

Service Security

- **Helmet**: Security headers on every Fastify service
- **Rate limiting**: 100 req/min per IP per service (`@fastify/rate-limit`)
- **CORS**: Only `localhost:3000` and production domain allowed
- **SSRF**: Food search proxy validates region against `ALLOWED_REGIONS` allowlist
- **Input validation**: Zod schemas on all route handlers

13. Database & Migration Strategy

Database Structure

Each microservice has its own PostgreSQL schema (via Supabase) — no cross-service SQL joins are allowed. Cross-service data references use UUID strings only.

Migration Workflow

```bash
# Development — create new migration
npx prisma migrate dev --name add_my_feature
```

```
# Production — deploy pending migrations (SAFE, non-destructive)
npx prisma migrate deploy

# ✕ NEVER in production
npx prisma db push    # drops tables on large drift
```

Resolving Failed Migrations (P3009)

When a migration partially fails (e.g. if `db push` was used previously):

```bash
# Option A: mark as already applied (when tables exist from db push)
npx prisma migrate resolve --applied "<migration_name>"

# Option B: mark as rolled back (when migration needs to re-run)
npx prisma migrate resolve --rolled-back "<migration_name>"

# Then apply remaining migrations
npx prisma migrate deploy
```

Migration File Conventions

- All `CREATE TABLE` statements use `IF NOT EXISTS`
- All `ALTER TABLE ADD COLUMN` statements use `IF NOT EXISTS`
- Enums use `DO $$ BEGIN ... EXCEPTION WHEN duplicate_object THEN NULL; END $$;`
- This makes all migrations **idempotent** — safe to re-run on existing DBs

14. Event-Driven Communication

EventBus Abstraction (`@nightfuel/events`)

```typescript
// Publish
await eventBus.publish('nightfuel:meal:meal-logged', {
  eventId:         crypto.randomUUID(),
  eventType:       'meal.logged',
  producedAt:      new Date().toISOString(),
  producerService: 'meal-service',
  correlationId:   crypto.randomUUID(),
  userId,
  payload:         { totalCalories, totalProtein, ... }
});
```

```
// Subscribe
eventBus.subscribe('nightfuel:meal:meal-logged', async (event) => {
  await progressService.updateDailyAdherence(event.userId, event.payload);
});
```

Event Catalogue

Event Topic	Producer	Consumers
`nightfuel:auth:user-registered`	auth-service	user-service, notification-service
`nightfuel:shift:shift-created`	shift-service	plan-service, circadian-engine
`nightfuel:shift:rotation-updated`	shift-service	plan-service
`nightfuel:meal:meal-logged`	meal-service	progress-service, notification-service
`nightfuel:plan:plan-generated`	plan-service	notification-service
`nightfuel:progress:streak-achieved`	progress-service	notification-service, community-service
`nightfuel:sleep:sleep-logged`	sleep-service	progress-service, circadian-engine

15. Deployment Architecture

Local Development

```bash
docker-compose -f infra/docker/docker-compose.yml up
```

Services started: PostgreSQL 16, Redis 7, all 14 microservices, Next.js web client.

Production — Railway

Each microservice is a separate Railway service. The monorepo Dockerfile for each service uses **Turborepo pruning** to bundle only the relevant package + its dependencies, keeping image sizes small.

Dockerfile pattern (per service):

```dockerfile
FROM node:22-alpine AS base

# Stage 1: Turborepo prune
FROM base AS pruner
RUN npm install -g turbo
COPY . .
RUN turbo prune meal-service --docker

# Stage 2: Install dependencies
FROM base AS installer
COPY --from=pruner /app/out/json/ .
RUN npm ci

# Stage 3: Build
FROM installer AS builder
COPY --from=pruner /app/out/full/ .
RUN npm run build

# Stage 4: Runtime
FROM node:22-alpine AS runtime
COPY --from=builder /app/services/meal-service/dist ./dist
COPY --from=builder /app/services/meal-service/prisma ./prisma
CMD ["sh", "-c", "npx prisma migrate deploy && node dist/index.js"]
```

Environment Secrets (Railway)

All secrets are Railway environment variables — never committed to git:

- `JWT_SECRET`
- `*_DATABASE_URL` (Supabase connection strings)
- `REDIS_URL` (Upstash)
- `ANTHROPIC_API_KEY`

16. Environment Variables Reference

Variable	Service	Description
`JWT_SECRET`	All	JWT signing secret
`AUTH_DATABASE_URL`	auth-service	PostgreSQL connection

SHIFT_DATABASE_URL	shift-service	PostgreSQL connection
PLAN_DATABASE_URL	plan-service	PostgreSQL connection
MEAL_DATABASE_URL	meal-service	PostgreSQL connection
PROGRESS_DATABASE_URL	progress-service	PostgreSQL connection
USER_DATABASE_URL	user-service	PostgreSQL connection
SLEEP_DATABASE_URL	sleep-service	PostgreSQL connection
EXERCISE_DATABASE_URL	exercise-service	PostgreSQL connection
NOTIFICATION_DATABASE_URL	notification-service	PostgreSQL connection
COMMUNITY_DATABASE_URL	community-service	PostgreSQL connection
CHAT_DATABASE_URL	chat-service	PostgreSQL connection
REDIS_URL	All	Upstash Redis URL
ANTHROPIC_API_KEY	ai-pipeline	Claude API key
PLAN_SERVICE_URL	meal-service	Inter-service HTTP call
AUTH_PORT	auth-service	Port (default 3001)
SHIFT_PORT	shift-service	Port (default 3002)
PLAN_PORT	plan-service	Port (default 3005)
MEAL_PORT	meal-service	Port (default 3006)
PROGRESS_PORT	progress-service	Port (default 3007)
NOTIFICATION_PORT	notification-service	Port (default 3008)
USER_PORT	user-service	Port (default 3009)
SLEEP_PORT	sleep-service	Port (default 3010)
EXERCISE_PORT	exercise-service	Port (default 3011)
COMMUNITY_PORT	community-service	Port (default 3013)
CHAT_PORT	chat-service	Port (default 3014)

LOG_LEVEL	All	Pino log level (info/debug/warn)

17. Development Workflows

Adding a New Service

32. Create `services/new-service/` folder
33. Copy `package.json` structure from an existing service
34. Add Prisma schema at `prisma/schema.prisma`
35. Implement Fastify app in `src/index.ts`
36. Create routes plugin in `src/routes.ts`
37. Add to `docker-compose.yml`
38. Add rewrite in `clients/web/next.config.js`
39. Add types to `packages/@nightfuel/types/src/models.ts`

Adding a New API Endpoint

40. Define Zod schemas in `src/schemas.ts`
41. Add service method in `src/[name].service.ts`
42. Register route in `src/routes.ts` with `preHandler: [authenticate]`
43. Add to `@nightfuel/types` if response shape is shared
44. Call from web client via `axios` instance in `clients/web/lib/api.ts`

Schema Changes (Database)

```bash
# 1. Edit prisma/schema.prisma
# 2. Create migration
npx prisma migrate dev --name your_change_description
# 3. This generates migration SQL + applies it locally
# 4. Commit the new migration file
# 5. Production deployment runs prisma migrate deploy automatically (via Dockerfile CMD)
```

TypeScript Strict Mode Patterns

The project uses `noUncheckedIndexedAccess: true`. Common patterns:

```typescript
// Array access — always might be undefined
const item = arr[0]!;                              // assert non-null
const item = arr.find(x => x.id === id) ?? arr[0]!; // fallback + assert

// Record access
```

```
const color = colorMap[key] ?? colorMap['default']!;

// String splitting
const date = d.toISOString().split('T')[0] as string;
```

18. Roadmap

Phase 1 — Core (Complete ✅)

- [x] Authentication & sessions
- [x] Shift scheduling
- [x] User profiles & onboarding
- [x] Meal logging (Open Food Facts)
- [x] Local food library (FoodDB — 760 foods)
- [x] Recipe management
- [x] Fasting tracker
- [x] Exercise library (ExerciseDB)
- [x] Progress tracking
- [x] Community forums
- [x] Coach chat (WebSocket)
- [x] Subscription tiers (Stripe)
- [x] Web client (Next.js 15)

Phase 2 — AI & Circadian (In Progress)

- [ ] Circadian engine fully wired to meal planner
- [ ] AI day plan generation (Claude integration)
- [ ] Meal adherence calculation (vs plan targets)
- [ ] AI weekly report generation
- [ ] Caffeine timing notifications

Phase 3 — Advanced Features

- [ ] Barcode scanning (camera API)
- [ ] Mobile app (React Native / Expo)
- [ ] Wearable integration (Apple Health, Google Fit)
- [ ] AI form correction guidance (computer vision)
- [ ] Ramadan mode fully implemented
- [ ] Social challenges with leaderboards
- [ ] Coach marketplace payments (Stripe Connect)

Phase 4 — Scale

- [ ] OpenTelemetry distributed tracing
- [ ] Kafka migration (from Redis Pub/Sub)

- [ ] React Native offline-first sync
- [ ] Multi-region deployment
- [ ] HIPAA compliance review (healthcare users)

NightFuel Project Design Document v2.0 — February 2026

Built with TypeScript, Python, Next.js 15, Fastify 5, Prisma 6, Redis, and Anthropic Claude